Conception Phase – Habit Tracking Application

1. Purpose and Scope

The purpose of this application is to design a modular habit tracking system that enables users to monitor and analyze recurring behaviors related to physical health, mental well-being, and personal development.

The initial system scope includes five predefined habits: Water Intake, Exercise (Walking), Sleep, Reading, and Meditation. These habits represent both quantitative and qualitative tracking scenarios, allowing validation of a flexible habit abstraction model.

2. Core Design Considerations and Acceptance Criteria.

The system design is guided by the following architectural principles:

- Separation of Concerns: Habit definitions, tracking data, business logic, and persistence are isolated into distinct components.
- Extensibility: The abstraction of habits ensures that new habit types can be introduced without structural redesign.
- Data Integrity: Persistent storage guarantees reliable historical tracking and accurate streak computation.
- Scalability: The data model supports long-term accumulation of entries without structural degradation.

3. Data Model and Core Classes

The system is structured around two primary domain entities:

- Habit: Represents the abstract definition of recurring behavior, including its name, frequency, and configuration parameters.
- HabitEntry: Represents a time-specific record associated with a Habit.

This separation ensures normalization of data and prevents redundancy. By isolating static habit definitions from dynamic tracking records, the system supports efficient analytics, historical queries, and long-term scalability.

4. Storage and Data Persistence

The Habit application utilizes SQLite as a lightweight relational database solution. SQLite is selected due to: Zero-configuration deployment, Full SQL support, Transactional integrity Compatibility with Python's standard library

This choice aligns with the project's console-based architecture while ensuring reliable persistent storage without introducing unnecessary system complexity.

Data retrieval is primarily date-based to enable daily, weekly, and monthly aggregation of habit performance metrics.

5. Requirements

Functional Requirements

- The system shall allow users to create and manage predefined habits.
- The system shall record habit completions on a daily or weekly basis.
- The system shall calculate and display habit streaks.
- The system shall persist all data across application sessions.
- The system shall provide analytical queries across stored habit data.

Non-Functional Requirements

- The system shall follow modular object-oriented design principles.
- The system shall ensure data consistency through transactional storage.
- The system shall support extension without modification of core logic.
- The system shall maintain a clear separation between user interface and business logic.

6. Habit-Specific Considerations

While all habits share a unified structural abstraction, behavioral variations exist at the configuration level. These differences are handled through parameterization rather than separate class implementations, preserving architectural consistency.

For example:

- Multi-entry habits (e.g., Water Intake, Meditation) permit multiple entries within a single period.
- Single-entry habits (e.g., Sleep) restrict tracking to one entry per defined period.
- Quantitative habits (e.g., water in liters, minutes of exercise) require numeric validation.
- Binary completion habits may require only a completion flag.

These distinctions are modeled using configurable attributes within the Habit class, such as frequency type, entry limit, and validation rules. This approach avoids subclass proliferation and maintains a clean, extensible design.

7. System Architecture and Modular Design

7.1. Presentation Layer (User Interface)

The system follows a layered architecture consisting of:

7.1.1 Presentation Layer (User Interface)

Implemented in main.py and cli.py, this layer is responsible exclusively for user interaction. It collects input, displays output, and forwards user requests to the controller. No business logic or database operations are implemented in this layer.

The CLI is currently the primary interaction mechanism. However, because business logic is isolated within the controller layer, the architecture allows future integration of alternative interfaces (e.g., GUI or web interface) without modifying core logic.

7.1.2. Application Layer (Controller)

The controller.py module contains the HabitController class, which orchestrates system behavior. It validates input, applies business rules, coordinates between domain objects and the persistence layer, and triggers analytical functions.

The controller acts as the central coordination unit, ensuring that presentation and data storage remain decoupled.

7.1.3. Domain Layer (Core Model)

The domain model consists of:

- Habit (habit definition and configuration)
- HabitEntry (time-specific completion record)

The one-to-many relationship between Habit and HabitEntry ensures normalization and prevents redundancy. Static habit definitions are separated from dynamic event data, enabling efficient querying and long-term scalability.

All business-relevant attributes (frequency, validation rules, entry limits) are encapsulated within the Habit class to avoid subclass proliferation and maintain extensibility.

7.1.4. Analytics Layer (Functional Programming)

The analytics module (analysis.py) contains pure functions responsible for computing:

- Longest and shortest streaks
- Average streak length
- Current active habits
- Performance aggregation

These functions operate independently of the user interface and database implementation. By isolating analytics logic, the system promotes reusability and supports independent testing.

7.1.5. Persistence Layer

The persistence layer consists of:

- repository.py (data access abstraction)
- database.py (SQLite connection and transaction handling)

SQLite was selected due to its zero-configuration setup, transactional integrity, and compatibility with Python’s standard library. All database interactions are encapsulated in the repository layer, preventing direct SQL access from higher layers.

This abstraction ensures that the database technology could be replaced in the future without affecting the domain or controller logic.

7.1.6. Testing Layer

Testing is integrated from the early design stage using pytest or unittest. The test suite validates:

- Habit creation and validation rules
- Streak calculation logic
- Analytics functions
- Repository operations

Dummy data and predefined four-week test datasets are used to simulate realistic scenarios and validate edge cases (e.g., skipped days, multi-entry limits).

By incorporating testing as an architectural component rather than an afterthought, the system improves reliability and reduces implementation risk.

7.2 Dependency Direction and Data Flow

The system follows a strict unidirectional dependency model:

Presentation → Controller → Domain / Analytics → Repository → Database

This prevents circular dependencies and ensures clear responsibility boundaries. Each layer depends only on the layer directly beneath it, increasing maintainability and facilitating future refactoring.

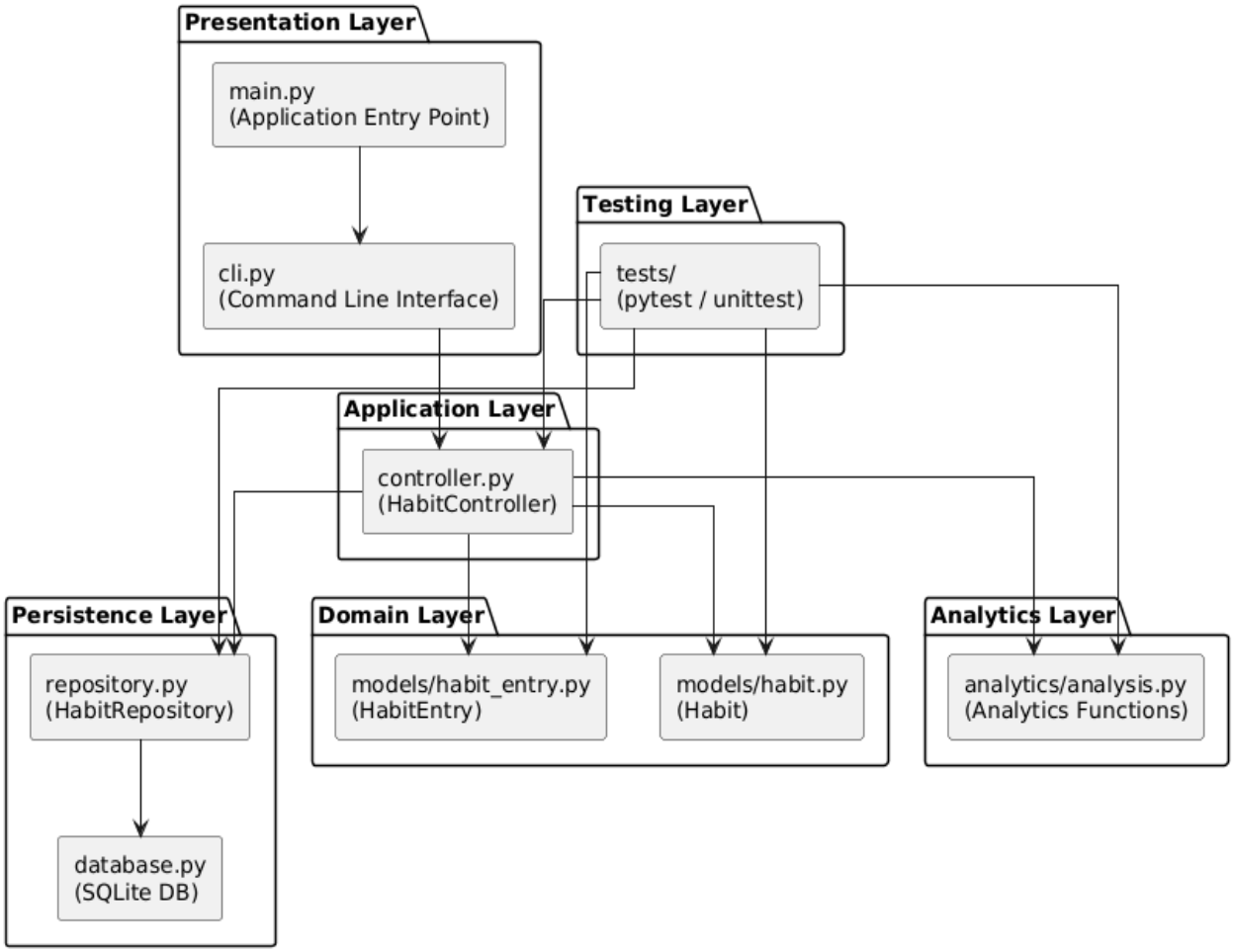
7.3 Architectural Rationale

The chosen architecture supports:

- Clear separation of concerns
- Extensibility without structural redesign
- Independent testing of components
- Replacement of the user interface without modifying business logic
- Scalability of stored data
- Long-term maintainability

By explicitly separating modules and defining clear responsibilities, the design mitigates common risks such as tight coupling, uncontrolled data access, and testing limitations.

This structured architectural foundation prepares the system for the implementation phase and ensures that future extensions can be introduced with minimal impact on existing components.



UML Component Diagram: *Figure 1: UML diagram illustrating the system architecture.*

8. Tools and Technologies

The system is implemented as a Python console-based system using object-oriented programming principles, supplemented with functional programming techniques for data analysis. The application will manage multiple habits, each with a defined task and schedule. Users can check off tasks at any point in time, and the system will track whether each habit is completed within its defined period. The application will calculate streaks, representing the number of consecutive periods a habit has been maintained without interruption. The stored habits can be analyzed to:

- Identify the user's longest and shortest habit streaks (maximum and minimum recorded).
- Calculate the average streak length across all habits.
- List the user's current daily habits.

- Determine which habits the user struggled with most in each period (based on frequency or performance).
- Aggregate streak data, including total, list of streaks, and grouped statistics.

9. Rationale Behind the Design

The chosen design minimizes redundancy, supports extensibility, and ensures a clear separation of concerns. This structure allows new features to be added with minimal impact on existing components.

11. Conclusion

Establishing a strong conceptual foundation is essential for successful implementation. The presented design supports the five initial habits while remaining flexible for future expansion, ensuring long-term usability and maintainability.